

Unidad II: De la abstracción a la implementación

Clase 2 – Pila y Cola: del contrato al código

Objetivo

Traducir una especificación abstracta a una implementación concreta respetando el contrato.

Tema

- TDA Pila y TDA Cola.
- Implementación estática con arreglos.
- Idea de representación interna.

Introducción: del concepto abstracto al código real

Cuando empezamos a programar, muchas veces creemos que lo más importante es escribir instrucciones que funcionen. Y sí, claro que importa. Pero en estructuras de datos aparece una idea más profunda: no alcanza con que el programa “ande”; también necesitamos que esté organizado, que sea entendible y que respete una lógica clara.

Ahí es donde aparece el **Tipo de Dato Abstracto**, o **TDA**.

Un TDA es una forma de pensar una estructura de datos desde su comportamiento. En otras palabras, primero definimos **qué puede hacer** una estructura, y recién después decidimos **cómo la vamos a construir internamente**. Esta separación es fundamental porque nos obliga a pensar como diseñadores de soluciones, no solo como personas que escriben código.

Imaginemos una situación cotidiana. En una cafetería, vos pedís un café desde una máquina automática. Tocás un botón, elegís el tipo de café y la máquina te lo entrega. Vos sabés qué hace la máquina: prepara café. Pero no necesitás conocer en detalle cómo circula el agua, cómo calienta la resistencia o cómo dosifica el café molido. Para usarla, te alcanza con entender su funcionamiento externo. Con los TDA pasa algo parecido: al usuario de la estructura le interesa saber qué operaciones puede realizar, no necesariamente cómo están implementadas internamente.

Esa es la gran idea de esta clase: aprender a distinguir entre el **contrato** y la **implementación**.

El contrato representa lo que la estructura promete hacer. Por ejemplo, si hablamos de una pila, sabemos que debe permitir agregar elementos, quitar el último agregado y consultar cuál está arriba. Eso pertenece al plano abstracto. La implementación, en

cambio, responde otra pregunta: ¿cómo logramos eso en Java? ¿Con un arreglo? ¿Con una lista? ¿Con índices? ¿Con variables auxiliares? Ahí entramos al plano concreto del código.

Este cambio de mirada es muy importante en programación. Muchas veces, cuando un estudiante ve un arreglo o una variable, cree que ya está viendo “la estructura”. Pero en realidad eso es solo una forma posible de representarla. La estructura, como idea, existe antes. Primero se piensa el comportamiento. Después se elige cómo representarlo.

Pensemos un ejemplo sencillo.

Supongamos que queremos modelar una pila de bandejas en un comedor. Si alguien agrega una bandeja, la coloca arriba de todo. Si alguien retira una bandeja, toma la que está arriba. No puede sacar la del medio sin desarmar la pila. Ese comportamiento ya define una lógica muy clara: el último en entrar es el primero en salir.

Eso es una pila.

Ahora bien, esa pila la podríamos implementar de muchas maneras. Podríamos usar un arreglo y una variable llamada tope, o podríamos usar otra estructura más compleja. Pero mientras el comportamiento siga siendo el mismo, el contrato no cambia.

Con una cola pasa algo diferente. Pensemos en una fila para subir al colectivo o para ser atendido en una ventanilla. La primera persona que llegó es la primera que será atendida. Si llega alguien nuevo, se ubica al final. Acá la lógica cambia: el primero en entrar es el primero en salir.

Eso es una cola.

Otra vez, la implementación puede variar. Pero el comportamiento esperado sigue siendo el mismo.

Entonces, cuando hablamos de TDA, estamos hablando de una forma ordenada de pensar:

primero definimos el comportamiento,
después elegimos la representación interna,
y finalmente escribimos el código respetando ese diseño.

Este enfoque trae una ventaja muy importante: permite construir programas más claros y más mantenibles. Si una estructura está bien encapsulada, quien la usa no depende de sus detalles internos. Eso significa que mañana podríamos cambiar parte de la implementación sin obligar a reescribir todo el programa. En otras palabras, el código queda menos atado a los detalles y más apoyado en ideas estables.

Acá aparece otro concepto clave: el encapsulamiento.

Encapsular significa proteger la información interna de una estructura y permitir el acceso solo a través de operaciones definidas. No se trata de esconder por esconder. Se trata de evitar que otros fragmentos del programa modifiquen la estructura de cualquier manera y generen inconsistencias.

Por ejemplo, si una pila tiene una variable interna llamada tope, no sería buena idea que cualquier parte del programa pudiera cambiarla libremente. Si eso pasara, la pila podría quedar en un estado inválido. En cambio, si obligamos a usar métodos como `push()` y `pop()`, mantenemos el control sobre cómo se modifica la estructura.

Esto tiene mucho valor en programación porque ayuda a que el sistema sea más confiable. La estructura cuida sus propios datos y garantiza que las operaciones respeten ciertas reglas.

En esta unidad, entonces, no solo vamos a estudiar estructuras. Vamos a estudiar una forma de pensar.

Vamos a pasar de una descripción conceptual a una implementación concreta. Vamos a ver que una estructura de datos no es solo “un arreglo con cosas adentro”, sino una organización que responde a un propósito y a un conjunto de reglas.

Y esto se relaciona directamente con los algoritmos.

Un algoritmo no trabaja en el vacío. Siempre necesita datos, y la forma en que esos datos se organizan cambia muchísimo la solución. No es lo mismo resolver un problema con una pila que con una cola. No es lo mismo acceder al último elemento agregado que respetar el orden de llegada. Por eso, aprender estructuras de datos también es aprender a elegir mejor cómo resolver problemas.

Punto de reflexión 1

Cuando usás una aplicación para escuchar música, ¿qué cosas ves vos como usuario y qué cosas suponés que ocurren internamente sin que las veas?

Punto de reflexión 2

Si una estructura funciona correctamente, ¿es necesario que quien la usa conozca todos sus detalles internos? ¿Por qué?

Punto de reflexión 3

¿Qué podría pasar si cualquier parte del programa pudiera modificar libremente las variables internas de una estructura?

Ejemplo para pensar juntos

Imaginemos un editor de texto. Cada vez que escribís, el sistema guarda acciones recientes para permitir “deshacer”. Cuando presionás `Ctrl+Z`, no borra cualquier acción al azar: borra la última que hiciste. Esa lógica se parece a una pila.

Ahora pensemos en una impresora compartida por varias personas. Los trabajos de impresión llegan y se atienden en orden. El primero que entró debería ser el primero en imprimirse. Esa lógica se parece a una cola.

Estos ejemplos muestran algo importante: las estructuras de datos no son inventos aislados del aula. Son modelos que aparecen una y otra vez en problemas reales.

TDA Pila (Stack): del concepto a la comprensión operativa

Cuando estudiamos estructuras de datos, no alcanza con memorizar nombres como pila, cola o lista. Lo importante es entender qué problema resuelve cada una, cómo se comporta y por qué ese comportamiento es útil para diseñar algoritmos. En este punto aparece la pila, una de las estructuras más simples de imaginar y, al mismo tiempo, una de las más importantes en programación.

Una pila es una estructura de datos en la que los elementos se agregan y se quitan siempre desde un mismo extremo. Ese extremo recibe el nombre de **tope**. Esto significa que no se puede sacar un elemento del medio ni acceder directamente al que quedó en la base. Toda la interacción ocurre desde arriba.

Si lo pensamos con una imagen cotidiana, una pila se parece mucho a una torre de platos, a una pila de libros o a una pila de bandejas. Cada nuevo elemento se coloca arriba del anterior, y cuando queremos retirar uno, sacamos el que quedó más arriba. Esa idea tan simple da origen al principio fundamental de esta estructura.

Principio de funcionamiento: LIFO

La pila responde al principio **LIFO**, sigla de *Last In, First Out*, que en español significa: **el último en entrar es el primero en salir**.

Esto quiere decir que el orden de salida no sigue el orden de llegada, sino el orden inverso. El elemento más reciente queda disponible primero, y el más antiguo queda al fondo.

Veamos una pequeña secuencia mental.

Si agregamos los valores 10, luego 20 y después 30, la pila queda así:

```
30 ← tope
20
10 ← base
```

Si ahora quitamos un elemento, no sale el 10, que fue el primero en ingresar. Sale el 30, porque es el último que entró y quedó en el tope.

Este comportamiento no es un detalle menor. Es la esencia misma de la pila. Si una estructura no respeta esta regla, entonces no estamos frente a una pila.

Nomenclatura básica de la pila

Para poder trabajar correctamente con este TDA, necesitamos conocer algunos términos básicos.

El **tope** o **cima** es el extremo de la pila donde se realizan todas las operaciones importantes. Ahí se agrega un nuevo elemento y también desde ahí se retira.

La **base** es el elemento que quedó en el fondo, es decir, el primero que ingresó y que permanece debajo de todos los demás.

Una **pila vacía** es una pila que no contiene ningún elemento. En ese estado, no se puede quitar ni consultar el tope porque simplemente no hay datos cargados.

Una **pila llena** aparece cuando la implementación tiene un tamaño máximo y ya no admite más elementos. Esto suele ocurrir cuando la representamos con un arreglo de longitud fija.

Una idea clave para recordar es la siguiente: en una pila, todo pasa en el tope; la base no se manipula directamente.

Las operaciones del contrato

Como todo TDA, la pila se define por las operaciones que ofrece. Estas operaciones forman parte de su contrato. Eso significa que describen qué puede hacer la estructura, aunque todavía no digamos cómo está implementada por dentro.

Las operaciones más importantes son las siguientes.

push(x)

Permite agregar un elemento x en el tope de la pila. Es una operación modificadora, porque cambia el contenido de la estructura.

pop()

Permite quitar el elemento que está en el tope. También es una operación modificadora. En muchas definiciones, además de quitarlo, lo devuelve como resultado.

peek() o top()

Permite consultar qué elemento está en el tope sin retirarlo. Es una operación observadora, porque permite mirar el estado de la pila sin modificarla.

isEmpty()

Indica si la pila está vacía. Devuelve un valor lógico: verdadero o falso.

isFull()

Indica si la pila está llena. Esta operación tiene sentido cuando la pila fue implementada con una capacidad fija, por ejemplo, con un arreglo.

Estas operaciones no están elegidas al azar. Son las que permiten trabajar con la pila respetando su lógica. No tendría sentido, por ejemplo, una operación que dijera “sacar el tercer elemento”, porque eso rompería la idea de acceso solo por el tope.

Notación básica de operaciones

En algoritmos y estructuras de datos, las operaciones suelen escribirse con una notación simple que ayuda a entender qué reciben y qué devuelven.

Por ejemplo:

`push(x)`

significa que la operación necesita un parámetro de entrada, en este caso `x`.

`pop()` -> elemento

significa que la operación no recibe parámetros, pero sí devuelve un elemento.

`isEmpty()` -> boolean

significa que la operación devuelve un valor lógico, es decir, verdadero o falso.

El símbolo de flecha se usa para indicar el retorno. Si una operación no devuelve nada, muchas veces eso se expresa como `void`.

Esta forma de escribir las operaciones es importante porque ayuda a leer el contrato del TDA de manera clara, antes de pasar al pseudocódigo o al lenguaje de programación.

Precondiciones: cuándo una operación puede ejecutarse

No todas las operaciones pueden aplicarse en cualquier momento. Algunas requieren que se cumplan ciertas condiciones previas. Esas condiciones se llaman **precondiciones**.

Por ejemplo:

`pop()` -> elemento

Precondición: la pila no debe estar vacía.

`peek()` -> elemento

Precondición: la pila no debe estar vacía.

`push(x)`

Precondición: la pila no debe estar llena, si tiene capacidad limitada.

Las precondiciones son muy importantes porque ayudan a evitar errores. No se trata solo de escribir código que compile, sino de pensar si la operación tiene sentido en el estado actual de la estructura.

En otras palabras, antes de hacer algo sobre una pila, conviene preguntarse: ¿está en condiciones de recibir esa operación?

Ejemplo simple de funcionamiento

Supongamos una pila vacía. Vamos a ejecutar las siguientes operaciones:

```
push(5)
push(8)
push(12)
```

Ahora la pila queda:

```
12 ← tope
8
5
```

Si hacemos peek(), obtenemos 12, pero la pila sigue igual.

Si hacemos pop(), sale el 12 y la pila queda:

```
8 ← tope
5
```

Si ahora volvemos a hacer pop(), sale el 8.

Queda finalmente:

```
5 ← tope y base
```

Este ejemplo muestra algo central: la pila no permite elegir libremente qué elemento retirar. Su comportamiento está completamente condicionado por la regla LIFO.

Casos de uso en la vida real de un programa

A veces los estudiantes piensan que las pilas son un contenido de aula sin relación con el mundo real. Sin embargo, aparecen todo el tiempo en sistemas y aplicaciones que usamos todos los días.

Un ejemplo muy conocido es la **navegación web**. Cuando visitamos una página y luego otra, y luego otra más, el navegador guarda ese recorrido. Cuando presionamos el botón “Atrás”, no vuelve a una página cualquiera: vuelve a la última que habíamos visitado. Es decir, recupera la más reciente. Esa lógica responde al comportamiento de una pila.

Otro caso fundamental es el de las **llamadas a funciones** dentro de un programa. Cuando una función llama a otra, y esa a otra más, el sistema necesita recordar en qué punto quedó cada una para poder retomar la ejecución correctamente. Esa información se organiza en una pila de ejecución. La última función llamada es la primera en finalizar. Por eso, el comportamiento natural vuelve a ser LIFO.

También aparece en funciones de **deshacer acciones**, como en un editor de texto. Si escribimos una palabra, luego borramos una línea y después cambiamos un formato, al usar “Deshacer” se revierte primero la última acción realizada. Otra vez, el último evento registrado es el primero en retirarse.

Por qué la pila es importante en algoritmos

La pila no es solo una estructura para guardar datos. Es una forma de organizar el acceso a esos datos. Y esa organización influye directamente en cómo resolvemos problemas.

Muchos algoritmos necesitan trabajar con el elemento más reciente, no con el más antiguo. En esos casos, la pila se vuelve una herramienta natural. Ayuda a controlar procesos, recorrer estructuras, deshacer pasos o administrar ejecuciones anidadas.

Aprender a reconocer cuándo un problema tiene lógica de pila es una habilidad valiosa. No se trata de forzar su uso, sino de detectar cuándo el comportamiento LIFO encaja con la situación.

Del TDA a la implementación

Hasta acá hablamos de la pila como idea abstracta. Es decir, describimos su comportamiento, su contrato, sus operaciones y su lógica general. El siguiente paso será llevar esa idea a una implementación concreta.

Cuando implementemos la pila con arreglos, necesitaremos decidir cómo representar internamente sus elementos y cómo controlar cuál es el tope. Ahí aparecerán variables auxiliares, validaciones y decisiones de diseño. Pero nada de eso cambia la esencia del TDA.

Esto es importante: una pila no deja de ser pila por estar implementada con un arreglo, con una lista enlazada o con otra técnica. Mientras conserve su contrato y su comportamiento LIFO, sigue siendo la misma estructura desde el punto de vista abstracto.

Para pensar

Pensá en una pila de objetos reales, como platos o libros. ¿Qué pasaría si quisieras sacar uno del medio sin tocar los demás? ¿Por qué eso rompe la lógica de la pila?

En una aplicación con botón “Atrás”, ¿por qué no tendría sentido usar una cola en lugar de una pila?

Cuando un programa ejecuta muchas funciones encadenadas, ¿por qué es útil que la última llamada sea la primera en resolverse?

TDA Cola (Queue): orden, atención y procesamiento secuencial

Después de estudiar la pila, aparece otra estructura fundamental en algoritmos y estructuras de datos: la **cola**. A simple vista puede parecer parecida, porque también almacena elementos y también tiene reglas para agregar y quitar. Sin embargo, su lógica de funcionamiento es distinta, y justamente ahí está su valor.

Una cola es una estructura de datos en la que los elementos se agregan por un extremo llamado **final** y se eliminan por el otro extremo llamado **frente**. Esto significa que no todo ocurre en un solo punto, como pasaba en la pila. En la cola hay dos extremos con funciones diferentes: uno sirve para incorporar nuevos elementos y el otro para retirarlos.

Si lo pensamos con una imagen cotidiana, una cola se parece a una fila de personas esperando para ser atendidas. Cuando llega una persona nueva, se ubica al final. Cuando comienza la atención, se atiende a la que está en el frente. Nadie debería “colarse” en el medio ni ser atendido antes que quien llegó primero. Ese comportamiento es el corazón del TDA Cola.

Principio de funcionamiento: FIFO

La cola responde al principio **FIFO**, sigla de *First In, First Out*, que significa: **el primero en entrar es el primero en salir**.

Este principio indica que la estructura respeta el orden de llegada. El elemento más antiguo, es decir, el que ingresó primero, es el primero que será retirado. Los nuevos elementos se van acumulando detrás de los anteriores.

Pensemos una pequeña secuencia.

Si agregamos los valores 10, luego 20 y después 30, la cola queda así:

Frente → 10, 20, 30 ← Final

Si ahora quitamos un elemento, sale el 10, porque fue el primero en entrar. Luego saldrá el 20 y después el 30.

Este comportamiento es exactamente el opuesto al de la pila. Mientras en la pila el último en entrar es el primero en salir, en la cola el orden se conserva.

Nomenclatura básica de la cola

Para trabajar correctamente con esta estructura, conviene dominar algunos términos básicos.

El **frente** es el extremo desde donde se eliminan los elementos. Es el punto de salida de la cola y representa al próximo elemento que será atendido o procesado.

El **final** es el extremo donde se agregan los nuevos elementos. Todo dato nuevo entra por ahí.

El **contenido de la cola** está formado por todos los elementos almacenados entre el frente y el final, respetando el orden de llegada.

Una **cola vacía** es una cola que no contiene elementos. En ese estado, no se puede quitar ningún dato ni consultar cuál está al frente.

Una **cola llena** aparece cuando la implementación tiene una capacidad máxima y ya no admite más inserciones. Esto suele ocurrir cuando la representamos con un arreglo de tamaño fijo.

Una idea sencilla para recordar es esta: en la cola, se entra por el final y se sale por el frente.

Las operaciones del contrato

Como todo TDA, la cola se define por sus operaciones. Estas operaciones forman el contrato de la estructura, es decir, describen qué se puede hacer con ella sin depender todavía de cómo será implementada internamente.

Las operaciones básicas son las siguientes.

enqueue(x) → void

Pre: la cola no está llena

Permite agregar un elemento x al final de la cola. Es la operación de inserción.

dequeue() → elemento

Pre: la cola no está vacía

Permite quitar el elemento ubicado en el frente de la cola. En muchas definiciones, además de quitarlo, lo devuelve como resultado.

front() → elemento

Pre: la cola no está vacía

Permite consultar cuál es el elemento que está en el frente sin eliminarlo.

isEmpty() → boolean

Indica si la cola está vacía.

isFull() → boolean

Indica si la cola alcanzó su capacidad máxima. Esta operación tiene sentido cuando la implementación es estática, por ejemplo, usando arreglos.

Estas operaciones reflejan algo importante: la cola no permite eliminar un elemento cualquiera ni insertar en cualquier posición. El acceso está regulado por su propia lógica. Eso es justamente lo que la convierte en una estructura útil y no en un simple conjunto de datos desordenados.

Notación básica de operaciones

En algoritmos y estructuras de datos, es habitual expresar las operaciones de forma compacta y precisa.

Por ejemplo:

enqueue(x) → void

indica que la operación recibe un parámetro x y no devuelve ningún valor.

dequeue() → elemento

indica que la operación no recibe parámetros, pero devuelve un elemento.

front() → elemento

indica que la operación devuelve el elemento que está adelante, sin modificar la estructura.

isEmpty() → boolean

indica que la operación responde con verdadero o falso.

El símbolo → representa el valor de retorno. Cuando una operación no devuelve nada, suele indicarse con void. Esta forma de notación es muy útil porque permite leer el contrato del TDA antes de pensar en la implementación concreta.

Precondiciones: cuándo una operación puede realizarse

No todas las operaciones pueden ejecutarse en cualquier momento. Algunas requieren que la estructura se encuentre en un estado válido. Esas condiciones previas se llaman **precondiciones**.

Por ejemplo:

dequeue() → elemento

Pre: la cola no está vacía

front() → elemento

Pre: la cola no está vacía

enqueue(x) → void

Pre: la cola no está llena

Las precondiciones ayudan a pensar correctamente el problema. No se trata solo de saber qué hace una operación, sino también de reconocer cuándo puede ejecutarse sin generar errores.

En la práctica, esto implica que antes de quitar o consultar, conviene verificar si la cola tiene elementos. Y antes de insertar, si existe un límite de capacidad, conviene verificar si todavía hay espacio disponible.

Ejemplo simple de funcionamiento

Supongamos que partimos de una cola vacía. Ejecutamos las siguientes operaciones:

enqueue(5)

enqueue(8)

enqueue(12)

La cola queda así:

Frente → 5, 8, 12 ← Final

Si ahora hacemos front(), obtenemos 5, pero la cola sigue igual.

Si hacemos dequeue(), sale el 5 y la cola queda:

Frente → 8, 12 ← Final

Si volvemos a hacer dequeue(), sale el 8.

Finalmente queda:

Frente → 12 ← Final

Este ejemplo muestra con claridad la idea central: la cola respeta el orden de llegada. No extrae el último elemento cargado, sino el primero que estaba esperando.

Casos de uso: turnos

Uno de los casos más fáciles de entender es el de los **turnos**. Pensemos en una fila en un banco, en una guardia médica, en una caja de supermercado o en una ventanilla de atención.

Cada persona que llega se ubica al final de la fila. La atención comienza desde el frente. La primera persona en llegar debería ser la primera en ser atendida. Esa lógica se corresponde exactamente con el comportamiento FIFO.

Este tipo de uso muestra por qué la cola es una estructura asociada a ideas de orden, justicia y secuencia temporal. No se trata solo de almacenar datos, sino de respetar el orden en que aparecieron.

Casos de uso: procesos en sistemas

La cola también aparece permanentemente dentro de los sistemas informáticos.

Un ejemplo clásico es la **cola de impresión**. Distintos usuarios envían documentos a una impresora compartida. Cada nuevo trabajo se agrega al final. La impresora atiende primero el documento que llegó antes. Así se evita el desorden y se mantiene una secuencia lógica.

Otro ejemplo es el **procesamiento de tareas** o solicitudes en un sistema. Un servidor puede recibir múltiples pedidos de usuarios. Si no puede atenderlos todos al mismo tiempo, los organiza en una cola y los procesa en orden. De esa manera, cada solicitud espera su turno.

También existen colas en sistemas operativos, en administración de procesos, en comunicaciones entre programas y en servicios que reciben eventos continuamente. En todos esos casos, la cola sirve para ordenar, administrar espera y distribuir trabajo.

Por qué la cola es importante en algoritmos

La cola no es solo una forma de guardar elementos. Es una manera de organizar la atención o el procesamiento de datos cuando el orden de llegada importa.

Muchos problemas del mundo real requieren justamente eso: que lo primero que llegó sea lo primero que se atienda. Por eso la cola aparece en simulaciones, sistemas de turnos, planificación de tareas, gestión de mensajes y procesos secuenciales.

Aprender a reconocer esta lógica es muy importante. Cuando un problema exige respetar el orden temporal de aparición de los datos, la cola suele ser una estructura natural para modelarlo.

Diferencia conceptual con la pila

Es importante no confundir ambas estructuras.

En la **pila**, todo sucede en el tope y el último en entrar es el primero en salir.

En la **cola**, se entra por el final, se sale por el frente y el primero en entrar es el primero en salir.

No es solo una diferencia de nombres. Cambia por completo la lógica de acceso. Esa diferencia hace que cada estructura sirva para problemas distintos.

Del TDA a la implementación

Hasta acá hablamos de la cola como una idea abstracta. Describimos su comportamiento, su contrato, sus operaciones y sus usos. El siguiente paso será implementar esa lógica en código.

Cuando la llevemos a una representación con arreglos, necesitaremos decidir cómo guardar los elementos y cómo controlar el frente y el final. Ahí aparecerán variables auxiliares, validaciones y decisiones de diseño. Pero todas esas decisiones deben respetar la esencia de la estructura.

Lo importante es recordar que la implementación puede cambiar, pero el contrato no. Mientras la estructura conserve el comportamiento FIFO, seguirá siendo una cola desde el punto de vista abstracto.

Para pensar

Si varias personas llegan a una fila y la última es atendida primero, ¿sigue siendo una cola? ¿Por qué?

¿Por qué una impresora compartida necesita respetar el orden de llegada de los documentos?

¿Qué diferencia conceptual hay entre consultar el frente de la cola y quitar el frente?

Del contrato a la implementación

Hasta ahora vimos a la pila y a la cola como **Tipos de Datos Abstractos**. Eso significa que los estudiamos desde lo que prometen hacer: agregar, quitar, consultar, verificar si están vacíos o llenos. En ese nivel todavía no hablamos de memoria, ni de arreglos, ni de índices. Hablamos del **comportamiento esperado**.

Pero en algún momento esa idea abstracta tiene que convertirse en un programa real. Y ahí aparece uno de los pasos más importantes en algoritmos y estructuras de datos: **traducir un contrato a una implementación concreta**.

Este punto es central porque marca el paso de “entender la estructura” a “construirla en código”.

El contrato no cambia, la implementación sí.

Eso quiere decir que una pila sigue siendo una pila aunque la implementemos con un arreglo, con nodos enlazados o con otra estrategia. Lo que no puede cambiar es su comportamiento: si deja de respetar LIFO, entonces ya no es una pila. Del mismo

modo, una cola puede representarse de distintas maneras, pero mientras conserve FIFO y sus operaciones básicas, sigue siendo una cola.

Entonces, cuando implementamos un TDA, no estamos inventando una estructura nueva. Lo que estamos haciendo es elegir **cómo representar internamente** ese comportamiento abstracto.

¿Qué significa traducir operaciones abstractas a código?

Cuando definimos un TDA, escribimos operaciones como estas:

```
push(x) -> void
pop() -> elemento
peek() -> elemento
isEmpty() -> boolean
```

o en una cola:

```
enqueue(x) -> void
dequeue() -> elemento
front() -> elemento
```

Estas operaciones expresan el contrato. Indican qué puede hacerse y, en algunos casos, bajo qué condición puede hacerse. Pero todavía no dicen cómo lograrlo en un lenguaje de programación.

Traducirlas a código significa tomar cada una de esas operaciones y convertirla en un **método concreto** dentro de una clase. Por ejemplo:

- push(x) pasa a ser un método que inserta un valor en una posición válida.
- pop() pasa a ser un método que recupera el elemento correcto, actualiza el estado interno y devuelve el dato.
- isEmpty() pasa a ser una verificación sobre alguna variable de control.

En otras palabras, la especificación abstracta se convierte en lógica programada.

La implementación necesita una representación interna

Un TDA abstracto no dice cómo están guardados físicamente los datos. Para que la estructura exista en memoria, tenemos que elegir una **representación interna**.

En esta unidad, la representación será **estática con arreglos**. Eso significa que los elementos van a almacenarse dentro de un vector de tamaño fijo.

Por ejemplo, una pila puede representarse internamente con:

- un arreglo para guardar los elementos
- una variable que indique cuál es el tope actual

Una cola puede representarse con:

- un arreglo para guardar los elementos
- una variable para el frente
- una variable para el final

Ese conjunto de datos internos constituye la forma concreta en la que la clase “vive” en memoria.

Esto es importante: el usuario de la estructura ve operaciones como `push()` o `enqueue()`, pero la clase internamente trabaja con índices, posiciones y validaciones.

Por eso se habla de **representación interna**: es la forma elegida para materializar la estructura sin alterar su comportamiento abstracto.

Estructura interna

La estructura interna está compuesta por los elementos que la clase necesita para funcionar por dentro.

En una implementación simple de una pila con arreglo, la estructura interna podría ser:

- un arreglo datos
- un entero tope

En una implementación simple de una cola, la estructura interna podría ser:

- un arreglo datos
- un entero frente
- un entero fin

Estos atributos no forman parte del contrato externo del TDA. Son recursos internos de la implementación. El usuario no debería modificarlos libremente, porque eso rompería el encapsulamiento.

La estructura interna responde a una pregunta práctica:

¿qué necesito guardar dentro de la clase para que sus operaciones funcionen correctamente?

La respuesta depende de cómo decidimos implementarla.

Variables de control

Además de almacenar los elementos, una estructura necesita saber en qué estado se encuentra. Para eso sirven las **variables de control**.

Las variables de control no representan datos del problema en sí, sino información necesaria para administrar la estructura.

En una pila, la variable de control más típica es el **tope**. Esa variable indica cuál es la última posición ocupada o, según la estrategia elegida, cuál es la próxima posición disponible.

En una cola, las variables de control suelen ser **frente** y **fin**. Con ellas se sabe desde dónde se quita un elemento y en qué posición se agrega el siguiente.

Estas variables son esenciales porque permiten responder preguntas como:

- ¿la estructura está vacía?
- ¿la estructura está llena?
- ¿dónde debo insertar?
- ¿qué elemento debo devolver?
- ¿qué posición debo actualizar después de quitar?

Sin estas variables, el arreglo sería solo un espacio de memoria con datos, pero no una estructura organizada.

Relación entre operaciones del TDA y métodos de la clase

Uno de los aprendizajes más importantes de esta parte es entender que cada operación del contrato debe reflejarse en la implementación.

Si el contrato dice que una pila debe permitir push, pop, peek, isEmpty e isFull, entonces la clase debería ofrecer métodos equivalentes.

Por ejemplo:

- push(x) del contrato se implementa como un método público push
- pop() del contrato se implementa como un método público pop
- isEmpty() del contrato se implementa como un método público que devuelve true o false

Esto muestra una relación muy directa entre el plano abstracto y el plano concreto.

El contrato funciona como una guía. Le dice a la implementación qué operaciones debe garantizar. La clase, por su parte, transforma esas operaciones en código ejecutable.

Lo importante no es solo “poner métodos con el mismo nombre”, sino lograr que cada uno respete el comportamiento prometido.

Por ejemplo, si pop() debe quitar el último elemento cargado, entonces el método implementado debe buscar exactamente ese elemento, devolverlo y actualizar la estructura para que siga siendo válida.

Del “qué” al “cómo”

Podemos pensar esta parte con una fórmula muy simple:

- El contrato dice **qué** debe hacer la estructura.
- La implementación decide **cómo** hacerlo.

Por ejemplo:

Contrato:

push(x) agrega un elemento al tope.

Implementación posible:

guardar x en la posición tope + 1 del arreglo y luego actualizar tope.

Otro ejemplo:

Contrato:

isEmpty() indica si la estructura no tiene elementos.

Implementación posible:

verificar si tope == -1 en la pila, o si frente y fin indican ausencia de elementos en la cola.

En ambos casos, el usuario no ve la mecánica interna. Solo ve el resultado de la operación.

La importancia del encapsulamiento en esta etapa

Cuando pasamos del contrato a la implementación, el encapsulamiento cobra todavía más sentido.

Si la estructura interna y las variables de control quedaran expuestas, cualquier parte del programa podría alterarlas sin respetar las reglas del TDA. Por ejemplo:

- cambiar el tope de una pila manualmente
- mover el frente de una cola sin quitar realmente un elemento
- escribir directamente en el arreglo ignorando la lógica de inserción

Eso generaría estructuras incoherentes y errores difíciles de detectar.

Por eso, al implementar un TDA, los atributos internos suelen declararse como **privados**, y las operaciones del contrato se ofrecen mediante métodos **públicos**.

Así, la clase conserva el control sobre su propio estado y garantiza que cualquier modificación pase por las reglas correctas.

Un mismo contrato, distintas implementaciones

Este es uno de los puntos conceptualmente más ricos de la materia.

Supongamos que una pila fue implementada primero con arreglos. Más adelante, podríamos decidir implementarla con nodos enlazados. En ambos casos, el usuario seguiría usando:

- push
- pop
- peek
- isEmpty

Desde afuera, la pila seguiría comportándose igual. Lo que cambió fue la técnica interna.

Esto demuestra por qué el contrato es más estable que la implementación. El contrato expresa la esencia del TDA. La implementación expresa una forma concreta de construirlo.

Por eso decimos: **el contrato no cambia, la implementación sí.**

Y esta idea no sirve solo para pilas y colas. Es una base muy importante de la programación orientada a objetos, del diseño modular y de la ingeniería de software en general.

Ejemplo conceptual: pila

Pensemos una pila abstracta con esta operación:

pop() -> elemento

Pre: la pila no está vacía

Esa especificación nos dice tres cosas:

- existe una operación llamada pop
- devuelve un elemento
- solo puede ejecutarse si hay datos cargados

Pero no nos dice cómo está construida la pila.

Cuando la implementamos con un arreglo, aparece el “cómo”:

- buscar el elemento ubicado en la posición marcada por tope
- guardarlo en una variable auxiliar
- decrementar tope
- devolver el valor recuperado

La operación abstracta es una sola, pero su realización concreta exige varios pasos internos.

Ejemplo conceptual: cola

Tomemos ahora una cola con esta operación:

enqueue(x) -> void

Pre: la cola no está llena

La especificación abstracta indica que debe agregarse un elemento x al final. Nada más.

Cuando la implementamos, tenemos que decidir:

- en qué posición del arreglo se guarda
- cómo se actualiza fin
- cómo sabemos si la cola estaba vacía
- cómo controlamos el límite de capacidad

Otra vez, el contrato marca el objetivo; la implementación resuelve la mecánica.

Errores comunes al pasar del contrato al código

En esta parte suelen aparecer varios errores típicos de aprendizaje.

Uno es confundir la estructura con el arreglo. El arreglo no es la pila ni la cola; es solo una posible representación.

Otro error es centrarse demasiado rápido en el código sin haber entendido bien el contrato. Cuando eso pasa, se implementa métodos, pero no siempre respeta el comportamiento esperado.

También es común olvidar actualizar correctamente las variables de control. Por ejemplo, insertar bien un elemento pero no mover el tope, o quitar un dato pero no avanzar el frente.

Por eso, antes de programar, conviene preguntarse siempre:

- ¿qué promete hacer esta operación?
- ¿qué estado interno necesito para lograrlo?
- ¿qué variable de control debo actualizar?
- ¿qué precondition debo verificar?

Implementación estática con arreglos: de la idea abstracta a la representación interna

Hasta este punto estudiamos la pila y la cola como Tipos de Datos Abstractos. Es decir, las entendimos desde su comportamiento: qué operaciones ofrecen, cómo se accede a sus elementos y qué principio respetan. La pila responde a LIFO y la cola a FIFO. Pero una estructura de datos no puede quedarse solo en la idea. Para que exista realmente en un programa, necesita una forma concreta de ser almacenada en memoria.

Ahí aparece la implementación estática con arreglos.

Cuando decimos que una estructura se implementa de forma estática, queremos decir que su capacidad se define de antemano y permanece fija durante toda la ejecución. En lugar de crecer o reducirse dinámicamente, la estructura trabaja con un bloque de memoria ya reservado. Ese bloque, en este caso, es un arreglo.

Un arreglo permite guardar varios elementos del mismo tipo en posiciones contiguas de memoria. Eso lo convierte en una buena base para construir pilas y colas simples, sobre todo en las primeras etapas de aprendizaje. Sin embargo, el arreglo por sí solo no define ninguna estructura. Lo que convierte a ese arreglo en una pila o en una cola es la lógica con la que se administra y las variables que controlan su uso.

Implementación estática de la pila

La pila es la más sencilla de representar con arreglos porque toda su lógica se concentra en un único extremo: el tope. No se necesita recorrer la estructura para insertar o quitar elementos. Todo ocurre siempre en la misma zona.

Para implementar una pila con arreglos se necesitan dos componentes básicos: el arreglo donde se almacenarán los datos y una variable de control llamada **tope**.

El arreglo guarda físicamente los elementos. La variable tope indica cuál es la última posición ocupada de la pila. Gracias a esa variable sabemos dónde insertar un nuevo elemento y cuál debe retirarse al desapilar.

La inicialización típica de una pila consiste en asignar al tope el valor -1.

Esto tiene una lógica muy clara. Como los arreglos comienzan en la posición 0, usar -1 indica que todavía no hay ningún elemento cargado. Es una manera de representar que la pila está vacía. En otras palabras, el tope todavía no apunta a una posición válida del arreglo.

A partir de esa base, las operaciones se entienden de forma muy natural.

Cuando se realiza un push, primero se incrementa el tope y luego se guarda el nuevo valor en esa posición. La lógica es: la pila crece hacia arriba y el nuevo elemento pasa a ocupar el nuevo tope.

Cuando se realiza un pop, se recupera el elemento ubicado en la posición señalada por el tope y luego se decrementa esa variable. Así, la pila “pierde” su último elemento y el tope pasa a señalar al anterior.

Esta implementación muestra una idea importante: no hace falta borrar físicamente el valor del arreglo al desapilar. Desde el punto de vista lógico, alcanza con mover el tope. Todo lo que quede por encima de él deja de pertenecer a la pila, aunque todavía exista en memoria.

La variable tope como eje de control

La variable tope es mucho más que un índice. Es la pieza que organiza toda la estructura. Permite saber si la pila está vacía, si está llena y cuál es su elemento actual más accesible.

Si el tope vale -1, la pila está vacía.

Si el tope llegó a la última posición del arreglo, la pila está llena.

Por eso, antes de ejecutar operaciones como push o pop, es necesario validar el estado de la estructura. Si intentamos apilar cuando no hay espacio disponible, o desapilar cuando la pila está vacía, el comportamiento sería incorrecto.

Esta necesidad de control muestra con claridad cómo una estructura no depende solo de almacenar datos, sino también de administrar correctamente su estado.

Ventajas de implementar una pila con arreglo

La implementación de la pila con arreglos tiene varias ventajas en el contexto de aprendizaje. Es simple de entender, fácil de programar y muy útil para visualizar cómo una idea abstracta se convierte en una solución concreta.

Además, las operaciones principales son muy eficientes. Tanto apilar como desapilar requieren solo unos pocos pasos y no dependen de cuántos elementos haya cargados. Esto la vuelve una estructura rápida y clara para problemas simples.

La principal limitación es que la capacidad queda fija desde el principio. Si el arreglo tiene tamaño 10, no se podrán cargar 11 elementos. Esa rigidez es justamente lo que caracteriza a una implementación estática.

Implementación estática de la cola

La cola también puede implementarse con arreglos, pero su lógica es un poco más compleja que la de la pila. La razón es sencilla: en la cola no todo ocurre en un solo extremo. Un elemento entra por un lado y sale por el otro.

Para representar una cola con arreglo se necesita el propio arreglo y, además, dos variables de control: **frente** y **fin**.

La variable frente indica la posición del primer elemento válido de la cola, es decir, el próximo que debe salir. La variable fin indica la posición del último elemento cargado, es decir, el punto donde se incorporará el siguiente elemento nuevo.

La inicialización más simple suele dejar frente en 0 y fin en -1, representando que la cola todavía no tiene elementos. En algunos diseños también se incorpora una variable cantidad, que ayuda a saber cuántos elementos contiene la estructura en cada momento.

La lógica básica de las operaciones también es bastante directa.

Cuando se hace un enqueue, se incrementa fin y se almacena el nuevo valor en esa posición. De ese modo, el elemento se agrega al final de la cola.

Cuando se hace un dequeue, se toma el valor ubicado en frente y luego se incrementa esa variable. Así, el primer elemento sale y el frente avanza hacia el siguiente.

A simple vista parece una solución tan simple como la de la pila. Sin embargo, acá aparece una dificultad importante.

El problema del espacio desperdiciado

En la implementación lineal de una cola con arreglo, cada vez que quitamos un elemento, la variable frente avanza. Eso significa que las posiciones iniciales del arreglo quedan libres. Pero el problema es que, en esta versión simple, esas posiciones no se reutilizan.

Por ejemplo, imaginemos una cola de tamaño 5. Si cargamos cinco elementos, el arreglo queda completo. Luego quitamos dos. En teoría, ahora hay dos lugares libres. Pero si fin ya llegó al final del arreglo, no podemos seguir insertando nuevos elementos sin una estrategia adicional, aunque al comienzo haya espacio disponible.

Ese fenómeno se conoce como **espacio desperdiciado**.

El arreglo tiene posiciones vacías, pero la implementación lineal no sabe volver a usarlas. Desde el punto de vista físico hay memoria libre; desde el punto de vista lógico, la cola parece haberse quedado sin lugar.

Este problema no aparece en la pila porque el crecimiento y la reducción ocurren siempre sobre el mismo extremo. En cambio, en la cola lineal, el frente y el fin avanzan en distintas direcciones lógicas y eso genera zonas inutilizadas.

Idea de cola circular

Para resolver el problema del espacio desperdiciado se introduce la idea de **cola circular**. La lógica general es sencilla: cuando el final del arreglo se alcanza, los índices vuelven al comienzo si allí hay posiciones libres.

De esta manera, el arreglo se aprovecha como si sus extremos estuvieran conectados. El último índice puede continuar en el primero, formando un recorrido circular.

En esta etapa no hace falta profundizar en la implementación matemática de esta solución, pero sí es importante que los alumnos entiendan el concepto: el problema de la cola lineal no es el arreglo en sí, sino la forma en que se recorren sus posiciones. La cola circular aparece justamente como una mejora para reutilizar espacios liberados y aprovechar mejor la memoria disponible.

Comparación entre la implementación de pila y cola

Aunque ambas estructuras pueden implementarse con arreglos, su administración interna es distinta.

En la pila, una sola variable basta para controlar toda la estructura. El tope resume el estado completo porque todo ocurre sobre un único extremo.

En la cola, en cambio, se necesitan al menos dos referencias lógicas: una para saber quién sale y otra para saber dónde entra el siguiente. Por eso la cola exige una gestión más cuidadosa del espacio.

Esta comparación ayuda a ver algo muy importante: el nivel abstracto puede parecer simple, pero cuando lo llevamos a código aparecen decisiones concretas que dependen del comportamiento de cada estructura.

Qué debemos comprender en esta parte

Lo esencial de esta unidad no es solo memorizar que la pila usa tope o que la cola usa frente y fin. Lo verdaderamente importante es comprender cómo una estructura abstracta necesita una representación interna y variables de control para poder existir en memoria sin perder su comportamiento original.

El arreglo es el soporte físico. Las variables de control organizan su uso. Y las operaciones del TDA se convierten en acciones concretas sobre esas posiciones.

En la pila, la lógica interna permite mantener el principio LIFO.

En la cola, la lógica interna permite mantener el principio FIFO.

Ese puente entre comportamiento abstracto y representación concreta es justamente uno de los aprendizajes más valiosos de la materia.

Para pensar

¿Por qué la pila puede controlarse con una sola variable y la cola necesita más de una?

Si una cola lineal deja espacios vacíos al principio del arreglo, ¿por qué no los reutiliza automáticamente?

¿Qué cambia si miramos una estructura desde el contrato y qué cambia si la miramos desde su implementación interna?

Manejo de errores y validaciones

Cuando implementamos una pila o una cola, no alcanza con programar las operaciones principales. También es necesario controlar en qué estado se encuentra la estructura antes de ejecutar cada acción. A eso lo llamamos validación.

En una **pila**, no se puede hacer `pop()` si está vacía, porque no hay ningún elemento para quitar. Tampoco se puede hacer `push()` si está llena, porque ya no hay espacio disponible en el arreglo.

En una **cola**, ocurre algo parecido. No se puede hacer `dequeue()` ni consultar el frente si la cola está vacía. Y no se puede agregar un nuevo elemento si la cola ya alcanzó su capacidad máxima.

Estas validaciones son importantes porque evitan errores lógicos y protegen la estructura. Antes de operar, el programa debe verificar si la acción tiene sentido en ese momento. De esta manera, no solo almacenamos datos, sino que también administramos correctamente el estado interno de la estructura.

Representación interna vs externa

Una de las ideas más importantes de esta unidad es distinguir entre cómo se ve una estructura desde afuera y cómo funciona por dentro.

Desde la **representación externa**, el usuario ve el TDA como un conjunto de operaciones. Por ejemplo, sabe que en una pila puede hacer push, pop o peek, y en una cola puede hacer enqueue o dequeue. En ese nivel, no necesita conocer detalles técnicos. Solo le importa qué puede hacer con la estructura.

Desde la **representación interna**, en cambio, aparece la implementación concreta. Ahí ya no hablamos solo de “agregar” o “quitar”, sino de arreglos, posiciones, índices y variables de control como tope, frente o fin.

Por ejemplo, para el usuario, hacer push(10) significa simplemente “agregar un elemento a la pila”. Pero para el sistema, eso implica verificar si hay espacio, incrementar la variable tope y guardar el valor en una posición específica del arreglo.

Esta diferencia es clave porque muestra que el TDA expresa el comportamiento visible, mientras que la implementación resuelve la mecánica interna. El usuario trabaja con la idea; el sistema trabaja con la representación concreta.

Encapsulamiento aplicado

Cuando implementamos una pila o una cola en una clase, no conviene que cualquier parte del programa pueda modificar directamente sus datos internos. Por eso, atributos como tope, array, frente o fin se definen como **privados**. De esta manera, solo la propia clase puede controlarlos.

En cambio, las operaciones del TDA, como push, pop, enqueue o dequeue, se ofrecen como **métodos públicos**. Así, el usuario puede trabajar con la estructura, pero siempre a través de las reglas correctas.

Esto protege la estructura interna y evita que quede en un estado inválido. El encapsulamiento, entonces, no solo oculta datos: también ayuda a mantener el orden y la coherencia de la implementación.

Comparación Pila vs Cola

Aunque ambas son estructuras lineales y pueden implementarse con arreglos, su lógica es distinta.

La **pila** sigue el principio **LIFO**: el último en entrar es el primero en salir. Por eso se usa en situaciones como navegación hacia atrás, llamadas a funciones o acciones de deshacer.

La **cola** sigue el principio **FIFO**: el primero en entrar es el primero en salir. Por eso se usa en turnos, impresión de documentos o procesamiento de tareas en orden.

También hay diferencias en la implementación. La pila se controla con una sola variable, tope, porque todo ocurre en un único extremo. La cola necesita al menos dos variables, frente y fin, porque un elemento entra por un lado y sale por el otro.

Resumen conceptual

Un TDA no se define por cómo está construido, sino por el comportamiento que ofrece. La pila y la cola son ejemplos simples, pero muy importantes, porque permiten entender cómo pasar de una idea abstracta a una implementación concreta.

Llevar un TDA a código implica tomar decisiones internas: elegir una estructura de representación, definir variables de control y programar métodos que respeten exactamente el contrato. En otras palabras, la esencia de la estructura está en su comportamiento; la implementación es la forma elegida para hacerlo funcionar.

Bibliografía

Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Pearson.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th ed.). Wiley.

Lafore, R. (2003). *Data Structures and Algorithms in Java* (2nd ed.). Sams Publishing.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Oracle. (2023). *The Java™ Tutorials*. <https://docs.oracle.com/javase/tutorial/>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.